Lab 30: Testing rules for test files spread across every folder

Topic 3: Claude Code Configuration & Workflows · Task 3.3: Apply path-specific rules for conditional convention loading

Runs on Simulator 3: Multi-Agent Research System · Run in class

Source: SkillCertPro practice exam Part 1, Q7 (unofficial third-party question)

Commit to an answer first

This team uses Claude Code to maintain the research system's TypeScript repository. Citation-verification tests, report-rendering tests, and orchestration tests live beside implementation files across multiple top-level folders. The team wants the same testing guidance to apply only when Claude Code is working on test files, without duplicating guidance in every package. What should they do?

A. Create .claude/rules/testing.md without frontmatter, then rely on the file name to limit it to test files.

B. Place the testing guidance in the root CLAUDE.md file so every Claude Code session receives the same instructions.

C. Add a CLAUDE.md file in each directory that contains tests, then repeat the same testing guidance in each file.

D. Create .claude/rules/testing.md with paths frontmatter matching the test-file patterns, then place the testing guidance below it.

Simulate the solution

```
# In the browser: open Simulator 3, then Lab 30
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 30

# In a terminal, from simulators/simulator-3-multi-agent-research/
uv run rules_sim.py D
uv run rules_sim.py
```

What you should see

D loads for the test edit only, one copy. A and B load always. C needs three copies.

A and B show the token on both edits; C shows it on both and needs three copies; D shows it only on the test-file edit with one copy.

Open `simulators/simulator-3-multi-agent-research/evidence/` and find it yourself.

The answer

D. One rule file with paths frontmatter matching the test patterns.

Claude Code separates broad project memory (CLAUDE.md or .claude/CLAUDE.md) from topic rules in .claude/rules/ . A rule file can carry YAML frontmatter with a paths field of glob patterns (for example "**/*.test.ts"), so one rule matches files by extension, directory or both. That fits conventions that cut across the tree, such as tests placed beside their code.

Why the others fail

A. A file name does not scope a rule. Without paths the rule loads unconditionally.

B. Avoids duplication but applies the guidance far more broadly than needed.

C. Directory CLAUDE.md files suit guidance that follows directory boundaries. Here they mean duplication and drift, which the team explicitly wants to avoid.

Check yourself

- D: token present on the test edit and absent on the implementation edit, one copy
- A and B: token present on both edits (filename and root placement do not scope)
- C: duplicate count equals the folder count (3) and the token appears for the implementation edit too

Now change one thing

In fixtures/sources.json replace the visual-arts TIMEOUT with [] and rerun: a valid empty result must not read as a failure.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.